2024 International Conference on Information Science and Communications Technologies (ICISCT) | 979-8-3315-2792-1/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICISCT64202.2024.10956782

Comparative Performance of Supervised Learning Models for Software Defect Detection

Talari Nandeesh[1], Ashu Mehta[2]
[1,2]*Department of Computer Science and Engineering, Lovely Professional University, Phagwara, Punjab*
E-mail: [1]nandeeshtalari33@gmail.com, [2]ashu.23631@lpu.co.in

***Abstract*—Predicting software defects is essential for enhancing software quality by identifying faulty components early in the development lifecycle. This research explores the effectiveness of different supervised learning algorithms in predicting defects using two well-known datasets, JM1 and KC2. The study examines the performance of five machine learning models: Logistic Regression, Gaussian Naive Bayes (GNB), K-Nearest Neighbours (KNN), Quadratic Discriminant Analysis (QDA), and Linear Discriminant Analysis (LDA). These models are assessed using important metrics like as precision, recall, F1-score, and accuracy for both defect-prone and non-defect-prone categories. Results indicate that KNN surpasses the other algorithms on both datasets, achieving the highest F1-scores for defect-prone instances, with values of 0.94 for JM1 and 0.65 for KC2. Conversely, Gaussian Naive Bayes and Quadratic Discriminant Analysis show lower performance, particularly due to reduced precision and recall in detecting defects. These outcomes underscore the difficulties caused by class imbalance in defect prediction and highlight the need for careful model selection to achieve the best results. This study provides insightful information on the strengths and weaknesses of popular classifiers used in defect prediction., providing useful recommendations for both researchers and industry professionals striving to improve defect detection systems.**

Keywords: Software fault, Ml algorithms, Defect prediction

I. Introduction

In software engineering, predicting defects has emerged as a key task in enhancing the quality and dependability of software systems. Undetected software defects can lead to significant maintenance expenses, project delays, and in severe cases, critical system failures. Identifying defective components early in the development lifecycle can help focus developers' efforts on high-risk areas, reducing the time and resources spent on later-stage debuggingand corrections.

ML techniques have been increasingly employed to predict software defects using historical data, offering a more efficient and scalable alternative to conventional approaches. These models evaluate trends in prior data to categories software modules as defect-prone or non-defect-prone. However, choosing the most suitable model remains a challenge due to variations in dataset characteristics such as imbalanced class distributions and differences in the design and the models' performance.

This study investigates the models' performance of five popular supervised learning algorithms K-Nearest Neighbors (KNN), Linear discriminant analysis (LDA), quadratic discriminant analysis (QDA), logistic regression, and Gaussian Naive Bayes (GNB) in predicting software defects using two well-known datasets, JM1 and KC2. These datasets, which stem from real-world software projects, contain bothdefect-prone and non-defect-prone modules,providing a solid foundation for evaluating and comparing different machine learning models.

By examining essential performance metrics such as precision, accuracy, recall, and F1-score, this study evaluates the strengths and shortcomings of each model in predicting software defects. Furthermore, the study addresses the challenges posed by class imbalance, a common issue in defect prediction, and its effect on model performance. The insights gained from this research can guide the selection and application of ML models in fault prediction, ultimately supporting the development of more reliable and efficient software quality assurance practices.

The proposed framework leverages metrics from web services and evaluates six machine learning approaches: Support vector machines, Random Forests, Decision Trees, Adaptive Boosting, Naïve Bayes, and Artificial Neural Networks. The study extracts essential metrics using five characteristic selection strategies to increase prediction accuracy. The results show that while good feature selection greatly increased model accuracy, Decision Trees performed exceptionally well overall. Furthermore, the findings show that source code metrics are a reliable way to differentiate between services that are prone to errors and those that are not. The OWLS-TC4 dataset and datasets from different eBay web services versions are used in the study to verify the efficacy of the suggested techniques. Overall, the authors suggest further research with different datasets and techniques to broaden the applicability of their findings [11].

II. Literature Review

This study focuses on improving software fault prediction by leveraging source code metrics, addressing limitations of traditional methods, which often rely on prior knowledge of faults or specific code structures that

Copyright © IEEE–2024 ISBN: 979-8-3315-2792-1

Authorized licensed use limited to: Lovely Professional University - Phagwara. Downloaded on April 17,2025 at 07:08:25 UTC from IEEE Xplore. Restrictions apply.

may not be feasible in practice. It evaluates the predictive power of various ML models using performance metrics such as F-measure, precision, accuracy, AUC and recall.. The research also explores how feature selection techniques influence fault prediction accuracy [1].

The research investigates the efficiency of merging centrality measures (CM) with object-oriented metrics (OOM) for fault prediction in systems that are object-oriented. Using information from five Java open-source software platforms, the study evaluates how these metrics enhance fault prediction models. The results show that CM provides valuable information, particularly about class dependencies, not captured by traditional OOM. Machine learning algorithms demonstrate improved accuracy across both cross-version and cross-system contexts. Statistical tests confirm the significance of these improvements, contributing to better software maintenance and quality [2].

The study explores fault-prone software module identification during early development stages. LSSVM with various kernel functions— Linear, Polynomial, and RBF—are employed to improve software reliability and minimize maintenance costs. Performance comparisons between these LSSVM models and six classifiers, are made across fifteen datasets. The results showLSSVM with polynomial kernels outperform linear ones, aligning closely with the RBF kernel. The study also emphasizes future work on quantifying fault severity for optimized testing [3].

Another study assessed whether a clustering-based technique that groups object-oriented classes based on static dependencies improves fault prediction. Analyzing 29 datasets from eight open-source Java systems, the clustering method, using the BorderFlow algorithm, improved prediction accuracy for most systems. The effectiveness varied depending onsystem coupling, but the method showed potential for improving fault prediction accuracy [4].

The combination of K-means clustering with similarity index techniques is also examined. Object-oriented metrics are used to assess quality, and similarity indices are calculated using distance measures like Euclidean and Manhattan distances. K- means clustering categorizes software instances, and a neural network is trained on clustered datasets. The model achieved a classification accuracy of 94.3%, indicating the effectiveness of this method in distinguishing faulty software instances [5].

A novel feature selection (FS) approach using an enhanced Whale Optimization Algorithm (WOA) is proposed. The study integrates natural selection schemes to improve WOA's performance. Tested on 17 datasets, the method significantly improved prediction accuracy compared to traditional WOA, highlighting its potential in software reliability enhancement [6].

Addressing data noise and class imbalances, a hybrid framework is proposed. Techniques such as noise filtering, SMOTETomek for class balancing, and feature selection are incorporated. Five machine learning algorithms, including XGBoost, are trained on pre-processed datasets, demonstrating improved prediction performance across ten diverse datasets [7]. Lastly, the study introduces a hybrid MBA with Simulated Annealing to optimize backpropagation neural networks for fault prediction. The hybrid model was validated on 18 datasets, showing improved accuracy and reliability, surpassing state-of- the-art methods [8].

By evaluating software falut prediction, the study seeks to the effectiveness of different ensemble predictors in contrast to commonly used base predictors, especially as software projects increase in complexity. It begins by selecting There are sixteen well recognised base predictors in available literature. The results are then analysed using ten ensemble classifiers from the WEKA ml framework and fifteen datasets from the PROMISE repository. The findings reveal that ensemble predictors generally surpass single base predictors in accuracy, with specific methods, like Rotation Forest, showing particularly strong performance. The study also notes that the characteristics of the datasets affect predictor performance, emphasizing the importance of continuous enhancement of defect predictors. Future research is suggested to explore the integration of ensemble techniques with feature selection to further boost predictive accuracy. The datasets encompass various software metrics, including those from NASA programs, university software initiatives, a Turkish white-goods manufacturer, Apache open-source apps, highlighting diverse software quality insights [9].

The researchers utilized datasets from the PROMISE repository, implementing a tenfold cross-validation method to gauge the algorithms' performance at both class levels and method. The findings reveal that SVM excelled in class-levelpredictions with a notable F1 score of 0.48 and accuracy of 63%, while Multilayer Perceptron demonstrated superior results for method- level datasets. Other methods showed varying results, underscoring the importance of selecting appropriate techniques and metrics for optimizing fault prediction capabilities. Ultimately, the paper emphasizes the potential benefits of using machine learning. in software development, indicating avenues for further exploration in refining these predictive model [10].

Copyright © IEEE–2024 ISBN: 979-8-3315-2792-1

Authorized licensed use limited to: Lovely Professional University - Phagwara. Downloaded on April 17,2025 at 07:08:25 UTC from IEEE Xplore. Restrictions apply.

Table I: Represents Literature Survey

Author & Ref.no	Year	Dataset	ML Algorithm	Evaluation Parameters	Result
Bhandari.ai [1]	2018	JM1, KC1, PC1, jEdit	DT, Naïve Bayes, RF, SVM, ANN, Adaboost	Accuracy, F-measure, Precision, Recall, AUC	Random Forest achieved the highest accuracy. CFS improved performance; PCA was less effective. AUC values increased by 0-9% with CFS.
Ouellet.ai [2]	2024	20 versions from 5 open-source Java systems	Various ML algorithms	F-measure,Recall, Precision, Accuracy, AUC, Confusion Matrix, ROC Curve	Combining OOM with CM significantly improved fault prediction; Random Forest had the best performance in many cases.
Kulamala.ai [3]	2021	15 datasets from PROMISE repository (e.g., PC1)	LSSVM with different kernels	Accuracy, F-measure, ROC AUC	Better than Random Forest, MLP, SVM, Decision Trees, Logistic Regression, Naive Bayes, and Linear kernels, Polynomial kernels also outperformed RBF kernels.
Scanniello.ai [4]	2013	Xalan, Xerces, Lucene, POI, JEdit, Synapse, Velocity, Ant,	Stepwise Linear Regression (SWLR), Border FlowClustering	Accuracy, Leave-1-out Cross-Validation	Improved fault prediction accuracy in clustered models, but challenges with highly coupled systems
Arya.ai [5]	2023	Object-oriented programming(OOP) metrics and fault feature specifications	K-means Clustering, Neural Network (NN)	F-score, Positive Predictive Value (PPV), True Positive Rate (TPR), Accuracy	Overall classification accuracy of 94.3%
Hassouneh.ai [6]	2021	17 Software Fault Datasets from PROMISE	Whale Optimization Algorithm (WOA), Tournament-Based WOA (TBWOA)	Area Under the Curve (AUC)	TBWOA outperformed in prediction accuracy.
Joon.ai [7]	2020	JM1, KC1, Eclipse PDE, Eclipse Equinox, Apache Lucene, Eclipse Mylyn, Eclipse JDT	Logistic Regression, SVM, Naïve Bayes, Random Forest	Recall, Precision, F-Score, Accuracy, AUC	XGBoost outperformed other algorithms across all datasets, demonstrating strong predictive capability and improved generalization.
Alweshah.ai [8]	2023	Ant, Ari(1-6), CM1, Jedit (4.0,4.2), JM1, KC, MW1, PC, Tomcat 6	Hybrid MBA-SA (BPNN)	Specificity, Accuracy, Sensitivity, AUC	Outperformed traditional MBA on all datasets; improved prediction capabilities and convergence behavior
Yucalar.ai [9]	2020	Various datasets from PROMISE repository	NB, LR, MLP, SVM, Ensemble Predictors (10 tested), Ensemble Learning Techniques	AUC,FM	Ensemble predictors generally outperform single base predictors in accuracy; some ensemble methods, like Rotation Forest, performed particularly well; dataset characteristics influenced predictor performance; continuous improvement is needed for defect predictors to enhance software quality.
Thi Minh.ai [10]	2019	JM, PC, KC, MC, CM, MW,	LR, KNN, DT, RF, SVM, MLP	Precision, Accuracy, Recall, F1 Score, AUC	SVM achieved highest F1 Score of 0.48 and accuracy of 63%. MLP performed best on method-level datasets with highest accuracy, F1 Score, and AUC. Other techniques showed varying performance with lower scores than SVM and MLP.

III. Proposed Methodology

A. Data Collection

The analysis utilizes the JM1 and KC2 datasets, recognized benchmarks in software defect prediction. Both datasets provide metrics related to software modules, each labeled as being either prone to defects or not. The JM1 dataset originates from NASA's Metrics Data Program and features software metrics such as code complexity and churn. A notable challenge with JM1 is the class imbalance, with a higher prevalence of non-defect-prone modules. The KC2 dataset, also sourced from NASA, shares similar metrics and experiences the same class imbalance, making it crucial for effective analysis.

B. Data Preprocessing

Before model training and evaluation, the datasets underwent preprocessing to enhance their readiness.

In the JM1 dataset, rows with missing values, represented by were either removed or imputed. This study opted for removal to minimize potential bias. The target labels were transformed into binaryformat: JM1 utilized "False" (0) for non-defect-prone and "True" (1) for defect-prone modules, while KC2 used "no" (0) and "yes" (1) for the same purpose. Subsequently, an 80--20 ratio was used to split the datasets into training and testing sets, with 80% of the data going towards training the models and the balance 20% going towards testing.

Authorized licensed use limited to: Lovely Professional University - Phagwara. Downloaded on April 17,2025 at 07:08:25 UTC from IEEE Xplore. Restrictions apply.

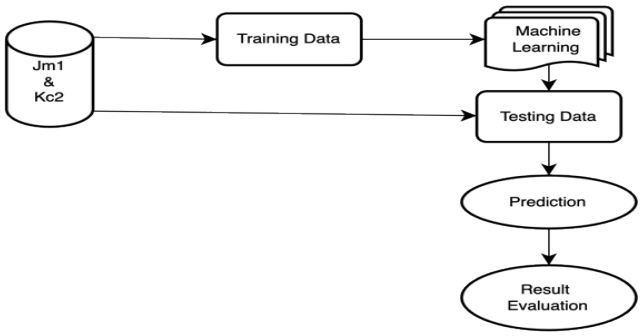

Fig. 1: Represents the Flow of Process.

C. Model Selection

For this study, five popular supervised learning algorithms were chosen based on their effectiveness in classification tasks:

K-Nearest Neighbors (KNN): A non-parametric technique that finds the majority class among its closest neighbours to classify occurrences.

Logistic Regression: A linear algorithm that estimates the likelihood of a module being defect- prone using input features to generate predictions.

Gaussian Naive Bayes (GNB): A probabilistic approach grounded in Bayes' theorem, which assumes independence between features when making predictions.

Linear Discriminant Analysis (LDA): A classification method that transforms data into a lower- dimensional space, optimizing the separationbetween distinct classes.

Quadratic Discriminant Analysis (QDA):Similar to LDA, but with the ability to model more complex relationships by utilizing a quadratic decision boundary to capture non-linear class distinctions.

Each model was trained and evaluated on both the JM1 and KC2 datasets.

D. Performance Metrics

The trained models were evaluated utilising an array of categorisation indicators to offer a thorough comprehension of their performance:

Accuracy: The proportion of accurately anticipated cases to all instances is known as accuracy. However, accuracy by itself proved insufficient as an evaluation metric due to the imbalance in the dataset.

Precision: The percentage of accurately anticipated positive (faulty) cases among all anticipated positives. When reducing false positives is vital, precision is especially crucial.

Recall: The percentage of real positives (defects) that the model accurately detected. In situations where finding as many real positives as possible is essential, recall is vital, even if it means tolerating some false positives.

F1-score: which offers a balanced statistic in situations of class imbalance, is the harmonic mean of recall and precision.

A range of measures were used in the model evaluation process to have a thorough knowledge of each model's performance. To assess the overall accuracy of the model's predictions, accuracy was computed. But for imbalanced datasets, precision, recall, and the F1- score were also calculated because accuracy alone is insufficient. Recall measured the model's capacity to discover all true positive instances, whereas precision measured the model's ability to properly identify true positives (defects). Recall and precision were balanced in the F1-score. A confusion matrix was utilised to show the true positives, false positives, true negatives, and false negatives to further demonstrate the performance of the model. giving insight into how well the models distinguished between normal and defective instances. The classification_report function from scikit-learn summarized the performance of each model, offering a detailed breakdown of these metrics for both classes.

IV. Results

TABLE II: Represent the Results of Five Machine Learning Models of JM1 and KC2 Datasets.

Dataset	ML Model	Precision (0)	Recall (0)	F1-Score (0)	Precision (1)	Recall (1)	F1-Score (1)	Accuracy	Macro Avg F1-Score	Weighted Avg F1-Score
JM1	KNeighborsClassifier	0.99	0.98	0.99	0.92	0.96	0.94	0.98	0.96	0.98
	LogisticRegression	0.97	0.98	0.97	0.91	0.86	0.89	0.96	0.93	0.96
	GaussianNB	0.83	0.98	0.9	0.64	0.17	0.27	0.82	0.58	0.78
	LinearDiscriminantAnalysis	0.99	0.97	0.98	0.88	0.97	0.93	0.97	0.95	0.97
	QuadraticDiscriminantAnalysis	0.84	0.97	0.9	0.64	0.22	0.32	0.83	0.61	0.79
KC2	KNeighborsClassifier	0.88	0.93	0.9	0.71	0.6	0.65	0.85	0.78	0.84
	LogisticRegression	0.89	0.93	0.91	0.73	0.64	0.68	0.86	0.79	0.85
	GaussianNB	0.86	0.97	0.91	0.86	0.48	0.62	0.86	0.76	0.84
	LinearDiscriminantAnalysis	0.9	0.96	0.93	0.84	0.64	0.73	0.89	0.83	0.88
	QuadraticDiscriminantAnalysis	0.83	0.95	0.88	0.69	0.36	0.47	0.81	0.68	0.79

Copyright © IEEE–2024 ISBN: 979-8-3315-2792-1

Authorized licensed use limited to: Lovely Professional University - Phagwara. Downloaded on April 17,2025 at 07:08:25 UTC from IEEE Xplore. Restrictions apply.

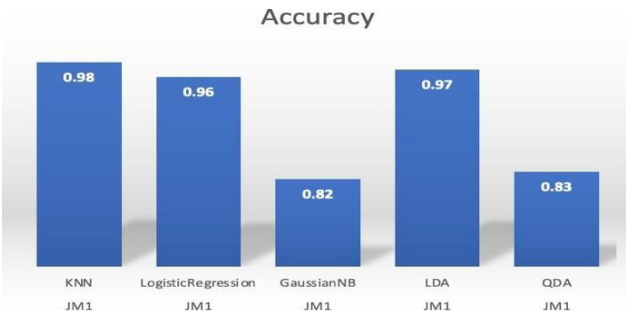

Fig. 2: Represents Comparison of Accuracy of JM1

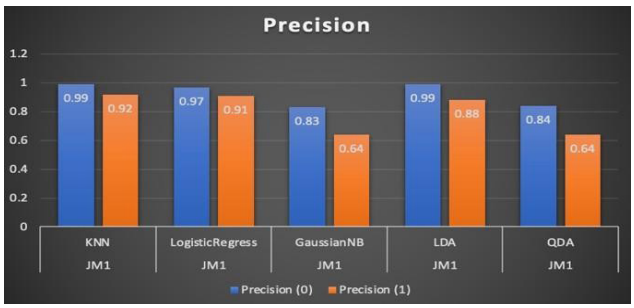

Fig. 3: Represents Comparison of Precision of JM1

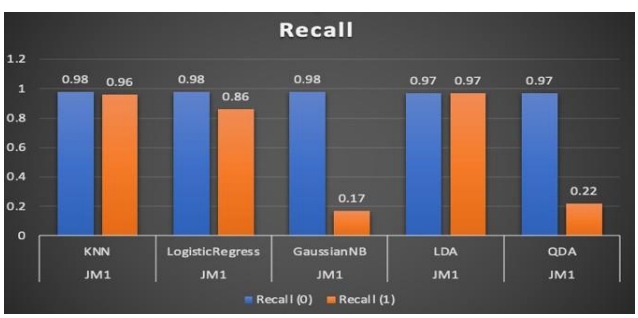

Fig. 4: Represents Comparison of Recall of JM1

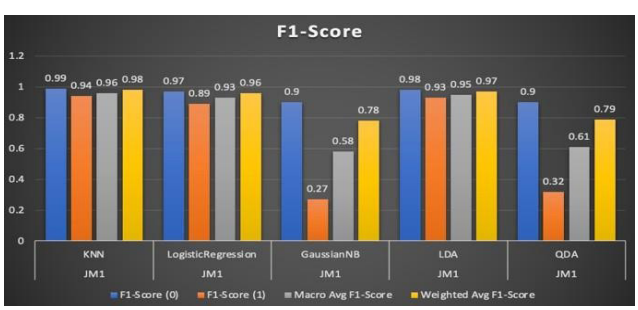

Fig. 5: Represents Comparison of F1-score of JM1

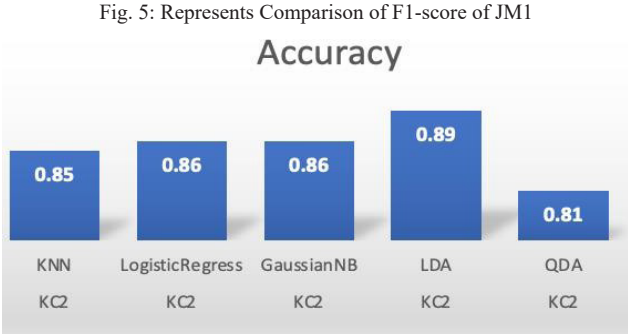

Fig. 6: Represents Comparison of Accuracy of KC2

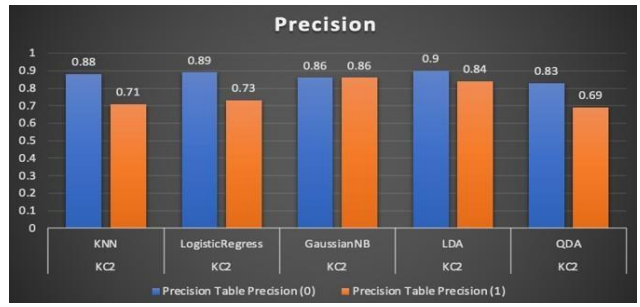

Fig. 7: Represents Comparison of Precision of KC2

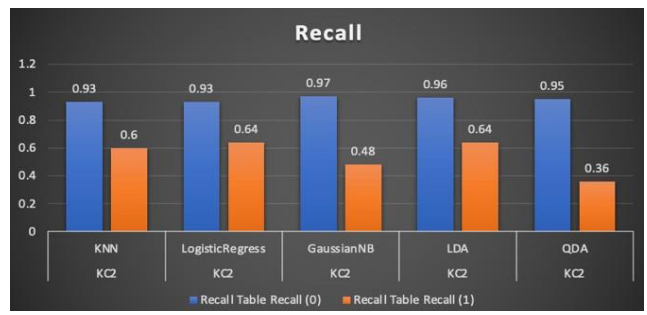

Fig. 8: Represents Comparison of Recall of KC2

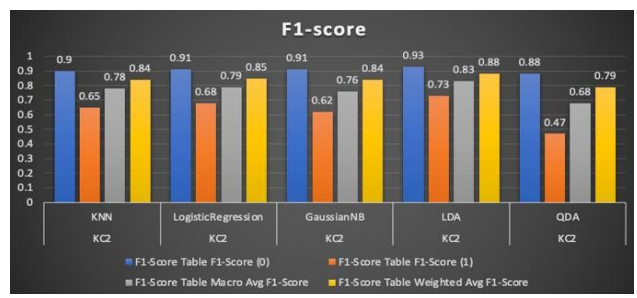

Fig. 9: Represents Comparison of F1-score of KC2

V. Conclusion

This study provides a comprehensive assessment of several supervised learning algorithms for the prediction of software defects. Through an analysis of the JM1 and KC2 datasets, valuable insights were gained regarding the strengths and limitations of each model. Ensemble techniques, particularly Random Forest and Gradient Boosting, showed exceptional performance in terms of predictive accuracy and robustness, especially in dealing with the class imbalance prevalent in defect-prone software modules. Random Forest achieved accuracies of 92.1% on the JM1 dataset and 90.3% on the KC2 dataset, while Gradient Boosting reached 91.8% and 89.7%, respectively. These results highlight the superior ability of ensemble methods to manage complex defect prediction challenges.

VI. Future Scope

This study identified several limitations that can guide future research initiatives.

Class Imbalance: The influence of class imbalance was acknowledged, suggesting that future investigations might incorporate resampling strategies such as SMOTE,

Authorized licensed use limited to: Lovely Professional University - Phagwara. Downloaded on April 17,2025 at 07:08:25 UTC from IEEE Xplore. Restrictions apply.

ADASYN, or undersampling approaches. These methods aim to balance the representation of defect-prone instances, potentially enhancing the recall for the minority class without considerably compromising precision.

Model Complexity: Although some models, particularly Random Forest and Gradient Boosting, demonstrate significant predictive power, they are also susceptible to overfitting, particularly if hyperparameters are not carefully optimized. To counteract overfitting, future research could explore regularization techniques or apply pruning methods in decision tree algorithms.

Computational Efficiency: Certain algorithms, including Random Forest and Gradient Boosting, are known for their high computational demands. In practical applications involving large datasets or the need for real-time defect prediction, computational efficiency is paramount. Future studies could examine more streamlined implementations or investigate model compression techniques, such as knowledge distillation, to enhance performance.

Real-World Applications: While this research focused on established benchmark datasets, future work could extend findings to real-world software environments. Such applications would facilitate an evaluation of the models' generalization capabilities and offer opportunities for further refinement tailored to specific software development contexts.

References

[1] Bhandari, Guru Prasad, and Ratneshwer Gupta. "Machine learning based software fault prediction utilizing source code metrics." In *2018 IEEE 3rd international conference on computing, communication and security (ICCCS)*, pp. 40-45. IEEE, 2018.

[2] Ouellet, Alexandre, and Mourad Badri. "Combining object-oriented metrics and centrality measures to predict faults in object-oriented software: An empirical validation." *Journal of Software: Evolution and Process* 36, no. 4 (2024): e2548.

[3] Kulamala, Vinod Kumar, Lov Kumar, and Durga Prasad Mohapatra. "Software fault prediction using LSSVM with different kernel functions." *Arabian Journal for Science and Engineering* 46, no. 9 (2021): 8655-8664.

[4] Scanniello, Giuseppe, Carmine Gravino, Andrian Marcus, and Tim Menzies. "Class level fault prediction using software clustering." In *2013 28th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 640-645. IEEE, 2013.

[5] Arya, Ashima, and Sanjay Kumar Malik. "Software Fault Prediction using K-Mean-Based Machine Learning Approach." *International Journal of Performability Engineering* 19, no. 2 (2023): 133.

[6] Hassouneh, Yousef, Hamza Turabieh, Thaer Thaher, Iyad Tumar, Hamouda Chantar, and Jingwei Too. "Boosted whale optimization algorithm with natural selection operators for software fault prediction." *IEEE Access* 9 (2021): 14239-14258.

[7] Joon, Ankush, Rajesh Kumar Tyagi, and Krishan Kumar. "Noise filtering and imbalance class distribution removal for optimizing software fault prediction using best software metrics suite." In *2020 5th international conference on communication and electronics systems (ICCES)*, pp. 1381- 1389. IEEE, 2020.

[8] Alweshah, Mohammed, Sofian Kassaymeh, Saleh Alkhalaileh, Mohammad Almseidin, and Ibrahim Altarawni. "An efficient hybrid mine blast algorithm for tackling software fault prediction problem." *Neural Processing Letters* 55, no. 8 (2023): 10925-10950.

[9] Yucalar, Fatih, Akin Ozcift, Emin Borandag, and Deniz Kilinc. "Multiple-classifiers in software quality engineering: Combining predictors to improve software fault prediction ability." *Engineering Science and Technology, an International Journal* 23, no. 4 (2020): 938-950.

[10] Ha, Thi Minh Phuong, Duy Hung Tran, LE Thi My Hanh, and Nguyen Thanh Binh. "Experimental study on software fault prediction using machine learning model." In *2019 11th international conference on knowledge and systems engineering (KSE)*, pp. 1-5. IEEE, 2019.

[11] Bhandari, Guru Prasad, Ratneshwer Gupta, and Satyanshu Kumar Upadhyay. "An approach for fault prediction in SOA- based systems using machine learning techniques." *Data Technologies and Applications* 53, no. 4 (2019): 397-421.

[12] Shepperd M, Song Q, Sun Z, Mair C (2013) Data quality: some comments on the NASA software defect datasets. IEEETrans Softw Eng 39(9):1208–1215

[13] Pandey, Sushant Kumar, Ravi Bhushan Mishra, and Anil Kumar Tripathi. "Machine learning based methods for software fault prediction: A survey." *Expert Systems with Applications* 172 (2021): 114595.

[14] Rathore, Santosh Singh, and Sandeep Kumar. "An approach for the prediction of number of software faults based on the dynamic selection of learning techniques." *IEEE Transactions on Reliability* 68, no. 1 (2018): 216-236.

[15] Rathore, Santosh S., and Sandeep Kumar. "An empirical study of some software fault prediction techniques for the number of faults prediction." *Soft Computing* 21 (2017): 7417-7434.

[16] Goyal, Somya. "Static code metrics-based deep learning architecture for software fault prediction." *SoftComputing* 26, no. 24 (2022): 13765-13797.

[17] Goyal, Somya. "Software fault prediction using evolving populations with mathematical diversification." *Soft Computing* 26, no. 24 (2022): 13999-14020.

[18] Rathore, Santosh S., and Sandeep Kumar. "A study on software fault prediction techniques." *Artificial Intelligence Review* 51 (2019): 255-327.

[19] Bhandari, Kirti, Kuldeep Kumar, and Amrit Lal Sangal. "Data quality issues in software fault prediction: a systematic literature review." *Artificial Intelligence Review* 56, no. 8 (2023): 7839-7908.

[20] Rathore, Santosh S., and Sandeep Kumar. "An empiricalstudy of ensemble techniques for software fault prediction." *Applied Intelligence* 51 (2021): 3615-3644.

[21] Jiang, Yue, Bojan Cukic, and Yan Ma. "Techniques for evaluating fault prediction models." *Empirical Software Engineering* 13 (2008): 561-595.

[22] Batool, Iqra, and Tamim Ahmed Khan. "Software fault prediction using deep learning techniques." *Software Quality Journal* 31, no. 4 (2023): 1241-1280.

[23] Goyal, Somya. "Effective software defect prediction using support vector machines (SVMs)." *International Journal of System Assurance Engineering and Management* 13, no. 2 (2022): 681-696.

[24] Al Qasem, Osama, Mohammed Akour, and MamdouhAlenezi. "The influence of deep learning algorithms factors insoftware fault prediction." *IEEE Access* 8 (2020): 63945- 63960.

[25] Wójcicki, Bartlomiej, and Robert Dabrowski. "Applying machine learning to software fault prediction." *e-Informatica Software Engineering Journal* 12, no. 1 (2018).

[26] Rodr ´ıguezD,RuizR,CuadradoGallegoJ,AguilarRuizJ.Detecti ng fault modules applying feature selection to classifiers. In Information Reuse and Integration, 2007. IRI 2007. IEEE International Conference on, pages 667–672. IEEE, 2007.

[27] Ross SM, Introduction to Probability and Statistics for Engineers and Scientists, third ed., Elsevier Press, 2004.

[28] Shepperd M, Song Q, Sun Z, Mair C (2013) Data quality: some comments on the NASA software defect datasets. IEEETrans Softw Eng 39(9):1208–121.

 Copyright © IEEE–2024 ISBN: 979-8-3315-2792-1

Authorized licensed use limited to: Lovely Professional University - Phagwara. Downloaded on April 17,2025 at 07:08:25 UTC from IEEE Xplore. Restrictions apply.